REOAGMS: Presentation script

This script gives the **exact words** each member says. The team deck `REOAGMS-Team-Presentation.pptx` has **10 slides** and runs together with the live console as one **10-minute** demo on one laptop. Each member speaks while their slide is up and switches to the app for the action shown under *On screen*. The same words are in the slides' speaker notes.

Before you start

- Stack running (`docker compose up -d`), browser signed in as `admin@reoagms.com` on http://localhost:5173, a second tab on Grafana. Zoom 100 %, notifications silenced.
- Two minutes earlier, open the Alerts page and resolve the open alerts; the simulator then raises fresh events within about a minute.
- Rehearse with a stopwatch. Hand over in one sentence, as written.
- If the fault has not arrived by 2:30, Govindu says "the simulator injects one every minute" and Thurunu uses the newest order already on the board.

Credentials

Use	Login
Console (all roles)	`admin@reoagms.com` , `manager@reoagms.com` , `operator@reoagms.com` , `engineer@reoagms.com` , `executive@reoagms.com` · password `Reoagms@2026`
Grafana http://localhost:3001	user `admin` · password `reoagms`
RabbitMQ http://localhost:15672	`reoagms` / `reoagms_password`
Prometheus http://localhost:9090	no login

Who does what

Member	Slides	Time on stage	Shows in the live demo
Charith	1, 2, 9	98 s	Login and Control room; Grafana
Nemal	3, 8	98 s	Assets page; executive read-only sign-in
Govindu	4	98 s	Live monitoring and the alert
Thurunu	5	98 s	Maintenance board
Yasiru	6	98 s	Notifications page
Sachinthana	7	98 s	Analytics report and CSV export
All members	10	12 s	Closing thank-you

Total spoken text: about 883 words, for a 10-minute slot at a calm pace, including the pauses for clicks. Each member's part below is complete: read only your own section.

The exact words, per member

Charith

Slide 1 · 0:00–0:15

On screen: Title slide

Charith says: Good morning, everyone. We are Group 9, and we are presenting REOAGMS, the Renewable Energy Operations, Asset and Grid Management System. It monitors and maintains renewable energy assets in real time.

Slide 2 · 0:15–1:13

On screen: Console: sign in as admin, then the Control room, pointing at the KPIs, the energy curve and the alert feed

Charith says: REOAGMS is six Spring Boot microservices behind one API gateway: identity, asset, monitoring, maintenance, notification and analytics. Each service owns its own PostgreSQL database and never reads another's, and they talk to each other through RabbitMQ events. The React console reaches everything through the gateway, which checks the token and the role on every request. I sign in as the administrator, and this is the control room: live generation, open alerts, device health and an hourly energy curve, all coming from real data in the databases. Nemal will show where that data comes from.

Slide 9 · 9:23–9:48

On screen: Grafana tab (REOAGMS Platform Health), then back to the slide

Charith says: Finally, operations. Grafana shows platform health from Prometheus, which scrapes all seven services. Sixty test methods protect the services, validation and error handling are built into every one of them, and one docker compose command starts all nineteen containers.

Nemal

Slide 3 · 1:13–2:08

On screen: Assets page: switch the three tabs, search, open Add, submit blank to show validation, then cancel

Nemal says: Thank you, Charith. The Asset Service owns the registry of everything we monitor. As you can see, there are five facilities, eleven assets and thirty-two sensors, covering solar, wind, hydro, battery storage and tidal. I switch between the facility, asset and sensor tabs, and search runs on the server with paging, so lists stay fast. Now I open the add form and leave the name blank: the form rejects it, because every input is validated. Other services refer to these assets only by ID. Govindu will now show these assets producing live data.

Slide 8 · 8:40–9:23

On screen: Sign out, sign in as executive@reoagms.com (read-only pages), then optionally operator@reoagms.com

Nemal says: Access is enforced at the gateway, and five roles carry different rights, as listed on this slide. I sign out and sign in as the executive. Every page is visible, but there are no action buttons, and the API itself returns forbidden for any write. Now the operator: it can handle alerts, but cannot create a facility. Passwords are stored as BCrypt hashes, and every login and user change is written to the audit log, which the administrator can review. Charith will close.

Govindu

Slide 4 · 2:08–3:46

On screen: Live monitoring page (tiles and dashed threshold), wait for the toast, then the Alerts page: acknowledge the alert

Govindu says: Thanks, Nemal. The Monitoring Service receives a reading from every sensor every few seconds and keeps the history: more than sixteen thousand readings are already stored. On the live monitoring page each tile is a real sensor, and the dashed line is its alert threshold. Every reading is checked against our seven alert rules. The simulator has just injected a fault, and here is the toast, instantly. On the Alerts page you can see the alert with its severity, the asset and the value that broke the threshold. Behind this, one alert is stored, and we keep only one open alert per rule and asset, so operators are never flooded. One event is published to RabbitMQ, and the Notification Service pushes it to this browser over WebSocket. The dashboard totals are cached in Redis for ten seconds, and if Redis ever stops, PostgreSQL answers instead. That same event is now reaching Thurunu's service.

Thurunu

Slide 5 · 3:46–5:24

On screen: Maintenance board: newest card, Assign a technician, Start, Complete, then show the history

Thurunu says: Yes. The Maintenance Service listens on its own RabbitMQ queue, received the same event and, because the severity is high, it opened a corrective work order by itself, with no one clicking anything. Here it is on the board, in the Created column. It stores the alert ID, so a repeated message never creates a duplicate, and it keeps one active corrective order per asset. Now I assign one of our four technicians. The card moves to Assigned. I start the work, and it moves to In progress. Then I complete it. Each step is its own endpoint, and the finished job is written to the maintenance history for that asset. So a fault becomes tracked, auditable work from the first reading to the last repair. Yasiru will show how people are informed.

Yasiru

Slide 6 · 5:24–7:02

On screen: Notifications page: point at PUSH delivered, open a message, use Retry on one entry

Yasiru says: The Notification Service read the same alert from its own durable RabbitMQ queue, so it never depends on the Monitoring Service being online. The consumer checks the event schema, then sends the message on each channel. In this history, the in-app push shows as delivered, and it reached this browser instantly. Every email attempt is stored with its status, its retry count and the reason, so nothing is hidden. If delivery fails, a scheduler retries up to three times, sixty seconds apart, and I can also retry any message from this page. Messages that keep failing go to a dead-letter queue, so no alert is ever lost. Email settings come from environment variables, so no credentials are in the code. Sachinthana will now turn all this data into reports.

Sachinthana

Slide 7 · 7:02–8:40

On screen: Analytics page: pick a facility, show the hourly energy chart and KPIs, Generate an energy report, click the CSV export

Sachinthana says: The Analytics Service calls the other services over REST and never touches their databases. For this facility, I pick it from the selector, and the chart shows hourly energy. Our figures are true kilowatt-hours: the service takes average power for each hour and integrates it over time. It also calculates availability, efficiency, capacity factor and utilisation, and compares the energy with what the rated capacity should have produced. Now I generate an energy report for the last day. It is stored, and I export it as CSV for management. We also keep six KPIs with targets, dashboards and performance reports. Executives see all of this read-only, and Nemal will show exactly who is allowed to see what.

All members

Slide 10 · 9:48–10:00

On screen: Thank you slide

All members says: Thank you very much. We are happy to take your questions.